

Design by Contract – Felipe Fontenelle

O artigo “Design by Contract”, de Bertrand Meyer, apresenta uma proposta muito interessante para aumentar a confiabilidade de softwares por meio de contratos formais entre partes do sistema. O autor utiliza a metáfora de contratos do mundo real para explicar como rotinas e módulos de software podem definir claramente suas responsabilidades, obrigações e garantias. A principal ideia do texto é que um software mais robusto depende de regras explícitas sobre o que cada componente deve receber e entregar.

Ao longo do artigo, Meyer critica o fato de que muitas abordagens tradicionais da programação focam apenas na flexibilidade e modularidade, mas deixam em segundo plano aspectos como corretude e robustez. Segundo ele, isso se torna ainda mais problemático em sistemas orientados a objetos, já que componentes reutilizáveis precisam ser extremamente confiáveis para funcionarem em diferentes aplicações.

O conceito central apresentado é o de “Design by Contract”. A ideia é que, sempre que uma rotina chama outra, exista um acordo entre cliente e fornecedor. O cliente deve garantir determinadas condições antes da chamada da função, enquanto o fornecedor garante um resultado após a execução. Essas regras são definidas por meio de pré-condições, pós-condições e invariantes.

As pré-condições representam o que precisa ser verdadeiro antes da execução de uma rotina. Já as pós-condições descrevem o que deve ser garantido após a execução. Além disso, existem os invariantes de classe, que representam propriedades que devem permanecer válidas durante todo o ciclo de vida de um objeto. O artigo mostra exemplos utilizando a linguagem Eiffel, criada pelo próprio Bertrand Meyer, demonstrando como essas verificações podem ser implementadas diretamente no código.

Um ponto que chamou bastante atenção foi a crítica feita à chamada programação defensiva. Em muitos projetos, programadores adicionam verificações redundantes em diversas partes do sistema para evitar erros. Meyer argumenta que isso aumenta desnecessariamente a complexidade do software, tornando-o mais difícil de manter e até menos confiável. Em vez disso, ele defende que cada responsabilidade deve estar claramente definida no contrato entre cliente e fornecedor. Assim, não há necessidade de repetir verificações em diferentes partes do código.

Outro aspecto importante discutido no artigo é o tratamento de exceções. É afirmado que exceções não devem ser vistas como situações normais do fluxo do programa, mas sim como sinais de falhas em contratos previamente definidos. O texto mostra que muitos mecanismos tradicionais de tratamento de exceções

acabam escondendo problemas ou permitindo que programas continuem executando mesmo após falhas críticas. O autor critica esse comportamento e propõe uma abordagem mais disciplinada e baseada em responsabilidade explícita entre os componentes do sistema.

O artigo também aborda o uso de monitoramento de assertions durante a execução do programa. Assertions são expressões booleanas utilizadas para verificar se as condições estabelecidas pelos contratos estão sendo respeitadas. Segundo o autor, elas são extremamente úteis para depuração e testes, pois ajudam a identificar erros de especificação, implementação ou uso incorreto de rotinas.

Muitas práticas modernas de desenvolvimento utilizam ideias semelhantes, mesmo sem aplicar formalmente o conceito de Design by Contract. Em APIs modernas, por exemplo, é comum definir claramente entradas esperadas, respostas possíveis e restrições de uso, algo muito parecido com as pré-condições e pós-condições descritas no artigo.

Além disso, o texto faz refletir sobre a importância da documentação e da clareza na comunicação entre partes de um sistema. Em projetos grandes, desenvolvidos por múltiplas equipes, definir contratos claros entre módulos pode reduzir erros de integração e facilitar manutenção futura. Também achei interessante a visão de que software correto não significa apenas funcionar em casos normais, mas também possuir comportamento previsível diante de falhas.

Apesar disso, alguns pontos podem parecer difíceis de aplicar completamente na prática. Em sistemas muito grandes e complexos, escrever e manter contratos detalhados para todas as rotinas pode demandar bastante tempo. Além disso, muitas linguagens populares atualmente não possuem suporte tão integrado para contratos quanto Eiffel. Mesmo assim, frameworks modernos e ferramentas de testes automatizados acabam implementando conceitos parecidos de forma indireta.